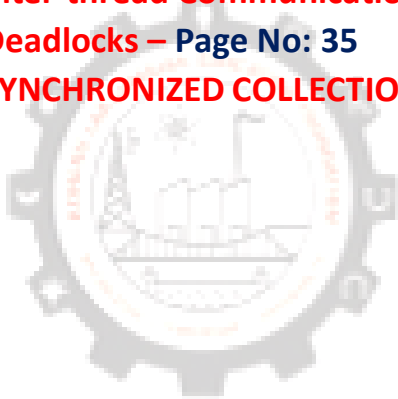


CO3 – Multithread in Java

Index:

1. Multi Programming, Multi Processing, Mukti tasking and Multi treading --Page No:2
2. Multithreading in Java –Page No: 5
3. Creating Threads in Java ---Page No:8
4. Life cycle of a Thread (Thread States) –Page No: 9
5. Thread class constructor – Page No: 11
6. Thread Methods -- Page No: 12
7. Thread Scheduler in Java: -- Page No: 13
8. . Concurrency In Java - Thread Pool and Executor Framework in Java – Page No:14
9. Thread Synchronization and Locks – Page No: 20
10. Inter-thread Communication in Java -- Page No: 27
11. Deadlocks – Page No: 35
12. SYNCHRONIZED COLLECTIONS & PARALLEL PROGRAMMING – PAGE No: 37



1. Multi Programming , Multi Processing, Multi tasking and Multi threading concepts:

1. Multiprogramming

Definition: Multiprogramming is a method where *multiple programs are loaded into memory and executed by the CPU one after the other*. The CPU switches between programs, giving the illusion that they are running simultaneously, although at any given time, only one program is actually executing.

Real-Time Example: Imagine you are listening to music on your computer while also working on a Word document and downloading a file from the internet. The operating system manages these tasks by keeping all three programs (music player, Word processor, and browser) in memory. The CPU switches between these tasks quickly, making it appear as though all are happening simultaneously.

Multiprogramming



2. Multitasking

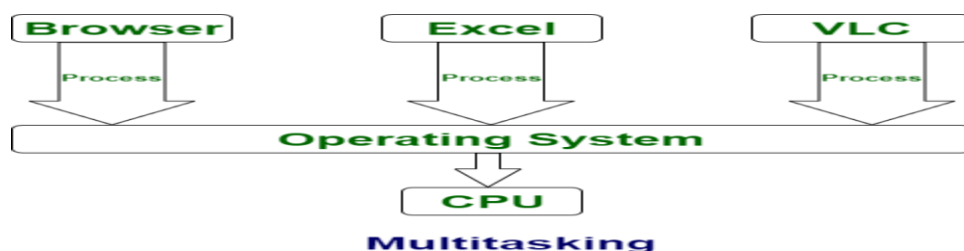
Definition: Multitasking is the ability of an operating system to execute multiple tasks (programs or threads) seemingly at the same time. This is achieved through time-sharing where the CPU rapidly switches between tasks so quickly that it appears as if the tasks are running simultaneously.

How It Works: In multitasking, the operating system divides CPU time into small segments, called **time slices**. Each task gets a time slice, and the CPU switches between tasks after each slice. This happens so fast (milliseconds) that users perceive that all tasks are running at the same time.

For Instance -

- The CPU executes part of Task A for a few milliseconds, then switches to Task B for another few milliseconds, and so on.
- If Task A needs to wait (e.g., for I/O), the CPU immediately switches to Task B without waiting.
- Preemptive multitasking is the most common type, where the operating system forcibly switches between tasks based on priority and time slice.

Real-Time Example: On your smartphone, you might be chatting with a friend on WhatsApp while also receiving emails in the background and listening to music. The operating system handles these tasks by giving each a small time slice, creating the appearance that they are all happening at once.



Benefits:

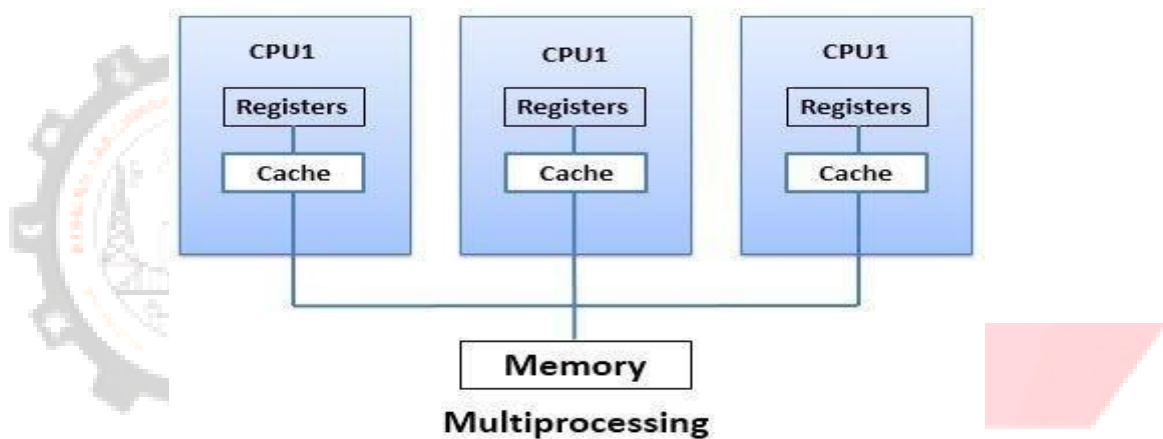
- Improve performance of CPU
- Multi-Tasking can be achieved by two ways.
 1. Process-based multi tasking (Multi-Processing) and
 2. Thread-based multi tasking.(Multi-Threading)

3. Multiprocessing

Definition: Multiprocessing refers to the ability of a system to use more than one processor (CPU cores) to execute multiple processes simultaneously. Each CPU can execute a different process at the same time.

- A multiprocessing system is one which has more than two processors.
- Multiprocessing is the ability for computers to complete multiple tasks at the same time without having to wait for one task to complete before the next task can be started. A *dual-core processor is twice as fast* as a single processor, and a *quad-core processor is four times as fast*.
- When one System connected to Multiple Processors in order to complete the task.

Real-Time Example: In modern computers with multi-core processors, each core can handle a separate task. For instance, while one core might be running a video game, another could be rendering a video, and a third might be scanning the system for viruses. All these tasks happen in parallel because each core is independently processing its own task.



- It is best suitable for system level performance.

4. Multithreading

Definition: Multithreading is a technique where a **single process** can **have multiple threads (sub process)** running concurrently, each performing a different part of the task. It allows for better utilization of CPU resources within a single application.

Real-Time Example: Consider a web browser that can load multiple web pages at once. Each tab represents a thread. While one thread is loading a video in one tab, another thread might be downloading images in a different tab, and yet another might be processing JavaScript for another webpage. These threads all belong to the same process (the web browser) but work on different tasks concurrently.

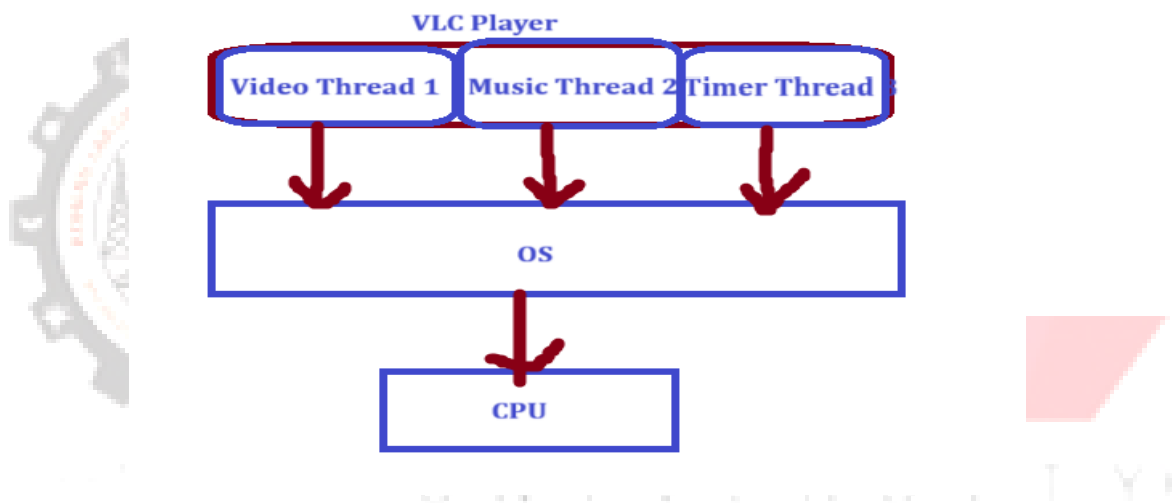
Benefits:

- Software
- Animations
- Games

Real Time - Example: Single Process (Gas Stove) with Four Burners (Threads)



Executing Multiple threads simultaneously



Difference between Multi programming, Multi tasking, Multi Processing and Multi threading:

Feature	Multiprogramming	Multitasking	Multiprocessing	Multithreading
Definition	Running multiple programs on a single CPU	Running multiple tasks (applications) on a single CPU	Running multiple processes on multiple CPUs (or cores)	Running multiple threads within a single task (application)
Resource Sharing	Resources (CPU, memory) are shared among programs	Resources (CPU, memory) are shared among tasks	Each process has its own set of resources (CPU, memory)	Resources (CPU, memory) are shared among threads
Scheduling	Uses round-robin or priority-based scheduling to allocate CPU time to programs	Uses priority-based or time-slicing scheduling to allocate CPU time to tasks	Each process can have its own scheduling algorithm	Uses priority-based or time-slicing scheduling to allocate CPU time to threads

Memory Management	Each program has its own memory space	Each task has its own memory space	Each process has its own memory space	Threads share memory space within a task
Context Switching	Requires a context switch to switch between programs	Requires a context switch to switch between tasks	Requires a context switch to switch between processes	Requires a context switch to switch between threads
Inter-Process Communication (IPC)	Uses message passing or shared memory for IPC	Uses message passing or shared memory for IPC	Uses inter-process communication mechanisms (e.g., pipes, sockets) for IPC	Uses thread synchronization mechanisms (e.g., locks, semaphores) for IPC

```

3 // code without Multi thread concept
4 public class ThreadExample1 {
5
6     public static void main(String[] args) {
7         Video v=new Video();
8         Music m=new Music();
9         Timer t=new Timer();
10        v.playVideo(); // Queue - 1 - first executes this method
11        m.playMusic(); // Queue - 2 - after first method ends second methods will start
12        t.showTimer(); /* Queue - 3 - after second method ends third methods will start -
13                        here next method have to wait till end of previous method
14                        - this is problem in this code */
15    }
16 }
17
18 class Video
19 {
20     void playVideo()
21     {
22     }
23 }
24
25
26 class Music
27 {
28     void playMusic()
29     {
30     }
31 }
32
33
34 class Timer
35 {
36     void showTimer()
37     {
38     }
39 }
40

```

- Multi threading is best suitable at programming level
- Java provides Pre-defined API for Multi-threading concept such as:

1. Thread class
2. Runnable Interface
3. ThreadGroup
4. Concurrency
5. ThreadPool

etc.....

2. Multithreading in Java:

1. Introduction to Multithreading

Multithreading is a process of executing multiple threads simultaneously. In Java, each thread is an independent path of execution, and it can run concurrently with other threads to perform tasks like file I/O, calculations, or background operations.

Threads in java:

What is Thread?

- **A thread is a lightweight subprocess, the smallest unit of processing. It is a separate path of execution.**
- Threads are independent. If there occurs exception in one thread, it doesn't affect other threads. It uses a shared memory area.
- However, we use multithreading than multiprocessing because threads use a shared memory area. They don't allocate separate memory area so saves memory, and context-switching between the threads takes less time than process.
- Java Multithreading is mostly used in games, animation, etc.

Advantages of Java Multithreading:

- 1) It **doesn't block the user** because threads are independent and you can perform multiple operations at the same time.
- 2) You can perform many operations together, so it saves time.
- 3) Threads are **independent**, so it doesn't affect other threads if an exception occurs in a single thread.

Multitasking

Multitasking is a process of executing multiple tasks simultaneously. We use multitasking to utilize the CPU. Multitasking can be achieved in two ways:

- Process-based Multitasking (Multiprocessing)
- Thread-based Multitasking (Multithreading)

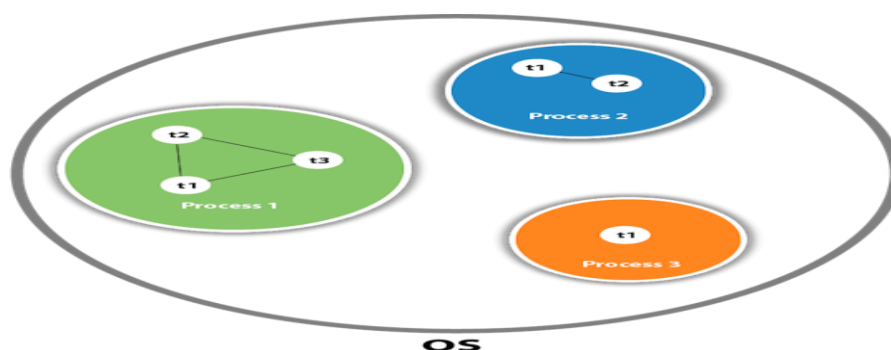
1) Process-based Multitasking (Multiprocessing)

- Each process has an address in memory. In other words, each process allocates a separate memory area.
- A process is heavyweight.
- Cost of communication between the process is high.
- Switching from one process to another requires some time for saving and loading [registers](#), memory maps, updating lists, etc.

2) Thread-based Multitasking (Multithreading)

- Threads share the same address space.
- A thread is lightweight.
- Cost of communication between the thread is low.

Note: At least one process is required for each thread.



As shown in the above figure, a thread is executed inside the process. There is context-switching between the threads. There can be multiple processes inside the [OS](#), and one process can have multiple threads.

Note: At a time one thread is executed only.

Java thread vs. Java process: what is the difference:

- A Java process is a program in execution. A Java thread is a subset of a Java process.
- A Java process consists of multiple threads and a Java thread is often regarded as a light-weight process.

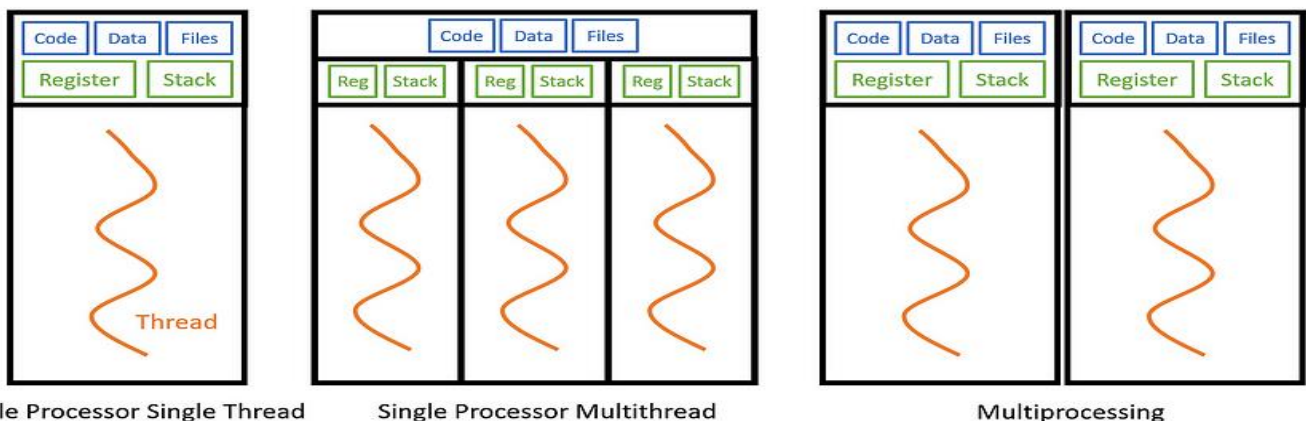
Real-Life Scenario to understand Process and Threads:

School:

- **Multiprocessing:** Imagine a school with multiple classrooms, each conducting different activities simultaneously. Each classroom represents a separate process. For instance, one classroom might be holding a math class, another might be teaching history, and a third might be conducting a science experiment. Each classroom operates independently, with its own teacher and resources. If there's an issue in one classroom, it won't affect the others.
- **Multithreading:** Alternatively, within a single classroom, there are different tasks being performed concurrently. For example, while the teacher is lecturing at the front of the class, students might be working on assignments, asking questions, or discussing topics in groups. These tasks can be seen as threads within the same process (classroom). They share resources such as the classroom space, textbooks, and the teacher's attention. Multithreading in this scenario allows for better utilization of resources and fosters collaboration among students.

Key Differences Between Multiprocessing and Multithreading

1. The key difference between multiprocessing and multithreading is that multiprocessing allows a system to have more than two CPUs added to the system whereas multithreading lets a process generate multiple threads to increase the computing speed of a system.
2. Multiprocessing system executes **multiple processes** simultaneously whereas, the multithreading system let execute **multiple threads** of a process simultaneously.
3. Creating a process can **consume time** and **even exhaust** the system resources. However creating threads is **economical** as threads belonging to the same process share the belongings of that process.



Single Processor Single Thread

Single Processor Multithread

Multiprocessing

3. Creating Threads in Java:

Threads can be created in two ways:

1. By extending Thread class
2. By implementing Runnable interface

Thread Class and Runnable Interface:

Thread Class: Java provides the Thread class, which represents a thread of execution in a program. To create a thread, you can either extend the Thread class or implement the Runnable interface.

Runnable Interface: The Runnable interface should be implemented by any class whose instances are intended to be executed by a thread. It has a single method, run().

Creating a Thread by Extending Thread Class

```

3 class MyThread extends Thread {
4     public void run() {
5         for (int i = 1; i <= 5; i++) {
6             System.out.println(Thread.currentThread().getName() + " - Value: " + i);
7             try {
8                 Thread.sleep(1000); // Sleep for 1 second
9             } catch (InterruptedException e) {
10                System.out.println(e);
11            }
12        }
13    }
14    public class ThreadExample3 {
15
16        public static void main(String[] args) {
17
18            MyThread t1 = new MyThread();
19            MyThread t2 = new MyThread();
20
21            t1.start(); // Start the thread
22            t2.start();
23        }
    }

```

Explanation: In this example, MyThread extends the Thread class and overrides the run() method. The start() method begins the execution of the thread, which in turn calls the run() method.

Creating a Thread by Implementing Runnable Interface

```

3 class MyRunnable implements Runnable {
4     public void run() {
5         for (int i = 1; i <= 5; i++) {
6             System.out.println(Thread.currentThread().getName() + " - Value: " + i);
7             try {
8                 Thread.sleep(1000); // Sleep for 1 second
9             } catch (InterruptedException e) {
10                System.out.println(e);
11            }
12        }
13    }
14    public class ThreadExample5 {
15
16        public static void main(String[] args) {
17            MyRunnable myRunnable = new MyRunnable();
18            Thread t1 = new Thread(myRunnable);
19            Thread t2 = new Thread(myRunnable);
20
21            t1.start(); // Start the thread
22            t2.start();
23        }
24    }
25 }

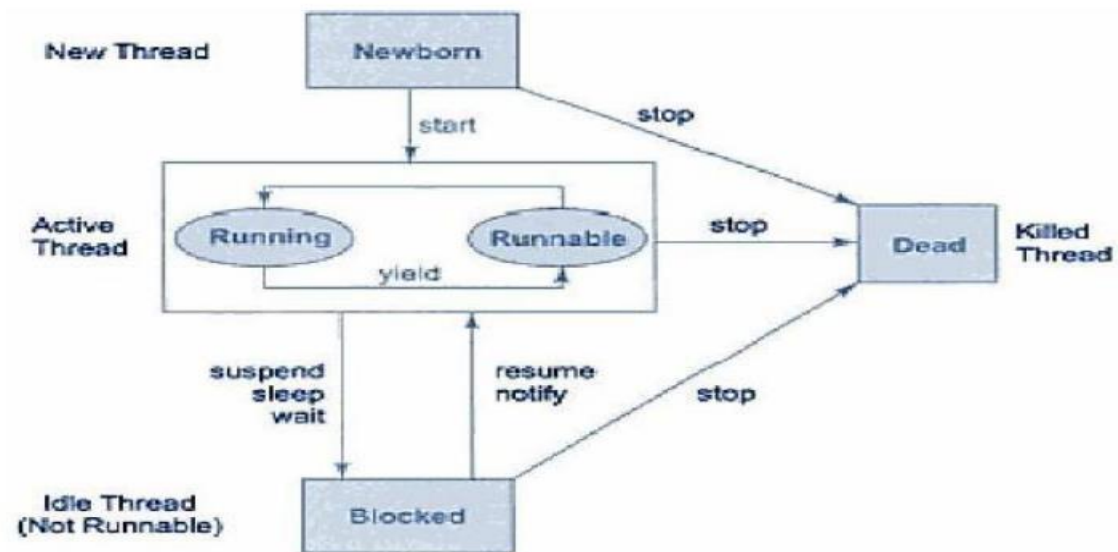
```

Explanation: MyRunnable implements the Runnable interface and overrides the run() method. A Thread object is created by passing an instance of MyRunnable to the Thread constructor.

4.Life cycle of a Thread (Thread States)

In Java, a thread can be in any of the five following states

- 1.Newborn State
- 2.Runnable State
- 3.Running State
4. Blocked State
5. Dead State



New: Whenever a new thread is created, it is always in the new state. For a thread in the new state, the code has not been run yet and thus has not begun its execution.

Active: When a thread invokes the start() method, it moves from the new state to the active state. The active state contains two states within it: one is **runnable**, and the other is **running**.

- **Runnable:** A thread, that is ready to run is then moved to the runnable state. In the runnable state, the thread may be running or may be ready to run at any given instant of time. It is the duty of the thread scheduler to provide the thread time to run, i.e., moving the thread the running state. A program implementing multithreading acquires a fixed slice of time to each individual thread. Each and every thread runs for a short span of time and when that allocated time slice is over, the thread voluntarily gives up the CPU to the other thread, so that the other threads can also run for their slice of time. Whenever such a scenario occurs, all those threads that are willing to run, waiting for their turn to run, lie in the runnable state. In the runnable state, there is a queue where the threads lie.
- **Running:** When the thread gets the CPU, it moves from the runnable to the running state. Generally, the most common change in the state of a thread is from runnable to running and again back to runnable.

Blocked or Waiting: Whenever a thread is inactive for a span of time (not permanently) then, either the thread is in the blocked state or is in the waiting state.

Example Program of Thread Life Cycle:

```

3  class ThreadOne extends Thread
4  {
5  public void run()
6  {
7      System.out.println("Thread One is Started");
8      for(int i=1;i<=5;i++)
9      {
10         System.out.println("Print i  "+i);
11         try
12         {
13             Thread.sleep(1000);
14         }
15         catch (InterruptedException e)
16         {
17             e.printStackTrace();
18         }
19     }
20     System.out.println("Thread one is completed");
21 }
22 }
23 class ThreadTwo extends Thread
24 {
25 public void run()
26 {
27     System.out.println("Thread two is started");
28     for(int i=1;i<=5;i++)
29     {
30         System.out.println("i  "+i);
31     }
32     System.out.print("Thread two is completed");
33 }
34 }
35
36 public class ThreadLifeCycleDemo {
37
38 public static void main(String[] args) {
39     ThreadOne t1 = new ThreadOne();
40     ThreadTwo t2 = new ThreadTwo();
41     t1.start();
42     t2.yield();
43     try
44     {
45         t1.sleep(1000);
46     }
47     catch (InterruptedException e)
48     {
49         e.printStackTrace();
50     }
51     t2.start();
52     System.out.println("Main thread end");
53 }
54 }

```

The Main Thread:

- When we run any java program, the program begins to execute its code starting from the main method.
- The JVM creates a thread to start executing the code present in main method. This thread is called as main thread.
- Although the main thread is automatically created, you can control it by obtaining a reference to it by calling `currentThread()` method.
- Two important things to know about main thread are,
 - It is the thread from which other threads will be produced.
 - main thread must be always the last thread to finish execution.

Example:

```
public class MainThread {

    public static void main(String[] args) {
        System.out.println("Main Thread...");
    }
}
```

5.Thread class constructor:

Thread class provide constructors and methods to create and perform operations on a thread. Thread class extends Object class and implements Runnable interface.

Commonly used Constructors of Thread class:

- `Thread()`
- `Thread(String name)`
- `Thread(Runnable r)`
- `Thread(Runnable r, String name)`

Example1: Thread(String name)

```
public class MyThread1 {

    public static void main(String[] args) {

        //Thread constructor with String as an argument
        Thread t1 = new Thread("kluThread");
        t1.start();
        String threadName = t1.getName();
        System.out.println("Thread Name:" + threadName);
    }
}
```

Example 2: Thread(Runnable r, String name)

```

public class MyThread2 implements Runnable {
    @Override
    public void run() {
        System.out.println("Thread is running....");
    }

    public static void main(String[] args) {

        //Creating MyThread2 object and assigning to Runnable interface reference variable
        Runnable runnable = new MyThread2();

        //Create Thread class object and pass the Runnable reference and String as an argument

        Thread t1 = new Thread(runnable, "KLUThread");
        t1.start();
        System.out.println(t1.getName());
    }
}

```

6.Thread Methods

1. **public void run():** is used to perform action for a thread.
2. **public void start():** starts the execution of the thread.JVM calls the run() method on the thread.
3. **public void sleep(long miliseconds):** Causes the currently executing thread to sleep (temporarily cease execution) for the specified number of milliseconds.
4. **public void join():** waits for a thread to die.
5. **public void join(long miliseconds):** waits for a thread to die for the specified miliseconds.
6. **public int getPriority():** returns the priority of the thread.
7. **public int setPriority(int priority):** changes the priority of the thread.
8. **public String getName():** returns the name of the thread.
9. **public void setName(String name):** changes the name of the thread.
10. **public Thread currentThread():** returns the reference of currently executing thread.
11. **public int getId():** returns the id of the thread.
12. **public Thread.State getState():** returns the state of the thread.
13. **public boolean isAlive():** tests if the thread is alive.
14. **public void yield():** causes the currently executing thread object to temporarily pause and allow other threads to execute.
15. **public void suspend():** is used to suspend the thread(depricated).
16. **public void resume():** is used to resume the suspended thread(depricated).
17. **public void stop():** is used to stop the thread(depricated).
18. **public boolean isDaemon():** tests if the thread is a daemon thread.(a tread which runs background)
19. **public void setDaemon(boolean b):** marks the thread as daemon or user thread.

20. **public void interrupt():** interrupts the thread.
21. **public boolean isInterrupted():** tests if the thread has been interrupted.
22. **public static boolean interrupted():** tests if the current thread has been interrupted.

Example: Thread Methods

```

2
3 class MyThreadTest extends Thread
4 {
5
6     @Override
7     public void run() {
8         System.out.println("Interrupted:" + Thread.currentThread().interrupted());
9
10        for(int i=1; i<10; i++) {
11            System.out.println(getName() + ":" + i);
12            try {
13                Thread.sleep(2000);
14            } catch (Exception e) {}
15        }
16    }
17 }

```

```

18 public class ThreadMethodsDemo {
19
20     public static void main(String[] args) {
21         MyThreadTest t1 = new MyThreadTest();
22         MyThreadTest t2 = new MyThreadTest();
23         // getName() and setName() and getId()
24         System.out.println("Thread 1 ID:" + t1.getId()); // deprecated from java
25         System.out.println("Thread 2 ID:" + t2.getId());
26         System.out.println("Thread 1 name:" + t1.getName());
27         System.out.println("Thread 2 name:" + t2.getName());
28         t1.setName("user1Thread");
29         t2.setName("user2Thread");
30         System.out.println("After changes Thread 1 name:" + t1.getName());
31         System.out.println("After changes Thread 2 name:" + t2.getName());
32
33         //setPriority() and getPriority() Note: Windows OS does not follow Thread priority
34         System.out.println("Priority of Thread 1:" + t1.getPriority()); // by default Thread priority is 5
35         System.out.println("Priority of Thread 2:" + t2.getPriority());
36         t1.setPriority(8);
37         t2.setPriority(4);
38         System.out.println("After changes the Priority of Thread 1:" + t1.getPriority());
39         System.out.println("After changes the Priority of Thread 2:" + t2.getPriority());
40
41         t1.start();
42
43         //to check thread is Alive or terminated
44         System.out.println("Thread 1 status:" + t1.isAlive());
45         // join method - let thread 2 wait until thread 1 complete its execution
46         try {
47             t1.join();
48         } catch (Exception e) {}
49         t2.start();
50         t2.yield();
51     }
52 }
53

```

7.Thread Scheduler in Java:

Thread Priorities:

Each thread has a priority. Priorities are represented by a number between 1 and 10. In most cases, the thread scheduler schedules the threads according to their priority (known as preemptive scheduling). But it is not guaranteed because it depends on JVM specification that which scheduling it chooses. Note that not only JVM a Java programmer can also assign the priorities of a thread explicitly in a Java program.

Priority: Priority of each thread lies between 1 to 10. If a thread has a higher priority, it means that thread has got a better chance of getting picked up by the thread scheduler.

Time of Arrival: Suppose two threads of the same priority enter the runnable state, then priority cannot be the factor to pick a thread from these two threads. In such a case, arrival time of thread is considered by the thread scheduler. A thread that arrived first gets the preference over the other threads.

Setter & Getter Method of Thread Priority:

public final int getPriority(): The java.lang.Thread.getPriority() method returns the priority of the given thread.

public final void setPriority(int newPriority): The java.lang.Thread.setPriority() method updates or assign the priority of the thread to newPriority. The method throws IllegalArgumentException if the value newPriority goes out of the range, which is 1 (minimum) to 10 (maximum).

There are three constants defined in Thread class:

- public static int MIN_PRIORITY (1)
- public static int NORM_PRIORITY (5)
- public static int MAX_PRIORITY (10)

Default priority of a thread is 5 (NORM_PRIORITY). The value of MIN_PRIORITY is 1 and the value of MAX_PRIORITY is 10.

Example Program:

```

3 class ThreadDemo extends Thread
4 {
5     @Override
6     public void run() {
7         System.out.println(getName() + ":" + "welcome to Thread Priority....");
8         try {
9             Thread.sleep(2000);
10        } catch (Exception e) {}
11    }
12 }
13 public class ThreadPriorityDemo {
14
15     public static void main(String[] args) {
16         ThreadDemo t1 = new ThreadDemo();
17         ThreadDemo t2 = new ThreadDemo();
18         ThreadDemo t3 = new ThreadDemo();
19
20         System.out.println("Priority of t1 Thread:" + t1.getPriority());
21         System.out.println("Priority of t2 Thread:" + t2.getPriority());
22         System.out.println("Priority of t3 Thread:" + t3.getPriority());
23         System.out.println("After changing Priorities:");
24         t1.setPriority(10);
25         t1.setPriority(3);
26         t1.setPriority(1);
27         System.out.println("Priority of t1 Thread:" + t1.getPriority());
28         System.out.println("Priority of t2 Thread:" + t2.getPriority());
29         System.out.println("Priority of t3 Thread:" + t3.getPriority());
30         System.out.println("Priority of the main thread is : " + Thread.currentThread().getPriority());
31         t1.start();
32         t2.start();
33         t3.start();
34    }
35 }

```

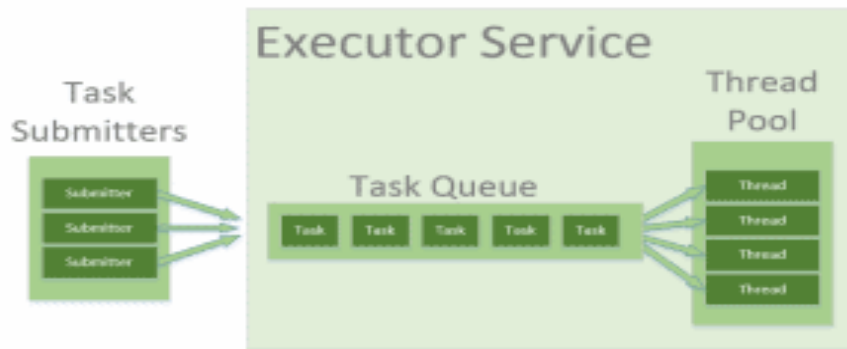
8. Concurrency In Java - Thread Pool and Executor Framework in Java

Java provides the Executor framework, which simplifies thread management by offering a mechanism to efficiently manage and reuse threads in the form of a thread pool. The thread pool reuses existing threads to execute multiple tasks instead of creating new threads for each task.

1. Thread Pool:

- A thread pool is a collection of threads (also called as Worker threads) that can be used to perform a set of tasks concurrently.
- These threads are used to execute tasks submitted to the pool, thus reducing the overhead of thread creation and destruction.

- When we use a thread pool, we write our concurrent code in the form of parallel tasks and submit them for execution to an instance of a thread pool. This instance controls several re-used threads for executing these tasks



2. Executor Framework:

- The Executor framework in Java provides a higher-level API for managing threads.
- It decouples the task submission from the actual mechanism used to execute the task, whether it's a single thread, a thread pool, or something else.

Thread Pools

A Thread Pool is a collection of pre-initialized threads that stand by ready to execute tasks. Instead of creating a new thread for each task, tasks are submitted to the pool, which assigns them to available threads. This approach offers several advantages:

Performance: Reusing threads reduces the overhead of thread creation and destruction.

Resource Management: Limits the number of concurrent threads, preventing resource exhaustion.

Task Management: Provides mechanisms to handle task queuing, scheduling, and execution policies.

What is the Executor Framework?

The Executor Framework is part of the `java.util.concurrent` package introduced in Java 5. It provides a high-level API for managing and controlling thread execution. The core interfaces and classes include:

Components of the Executor Framework:

- I. **Executor Interface:**
 - The root interface for executing tasks.
 - It has a single method:
`void execute(Runnable command);`
- II. **ExecutorService Interface:**
 - A sub-interface of `Executor`, it provides methods to manage the lifecycle of threads and obtain `Future` objects to track task completion.
- III. **ScheduledExecutorService Interface:**
 - A sub-interface of `ExecutorService`, used for scheduling tasks to run after a delay or periodically.
- IV. **ThreadPoolExecutor Class:**
 - Provides a flexible thread pool implementation.

V. Executors Utility Class:

- Provides factory methods for creating thread pools.

In order to use the executor framework, we have to create a thread pool for executing the task by submitting that task to that thread pool.

A framework having a bunch of components that are used for managing worker threads efficiently is referred to as Executor Framework. The Executor API reduces the execution of the task from the actual task to be executed through the Executors. The executor framework is an implementation of the Producer-Consumer pattern. class provides a set of methods for creating **ThreadPools** of worker threads.

Thread pools overcome this issue by keeping the threads alive and reusing the threads. Any excess tasks flowing in, that the threads in the pool can't handle are held in a Queue. Once any of the threads get free, they pick up the next task from this queue. This task queue is essentially unbounded for the out-of-box executors provided by the JDK.

Types of Thread Pools

Java's Executor Framework provides several types of thread pools, each suited for different scenarios.

a) FixedThreadPool

A fixed-size thread pool with a set number of threads. If all threads are busy, new tasks wait in the queue until a thread becomes available.

ExecutorService fixedThreadPool = Executors.newFixedThreadPool(int nThreads);

Example 1: Using FixedThreadPool

A fixed thread pool is useful when you know the number of tasks, and they are computationally expensive.

Scenario: Suppose you have a set of tasks (e.g., processing customer orders) that need to be executed concurrently, but you want to limit the number of threads to prevent resource exhaustion.

```

5
6 public class FixedThreadPoolExample {
7
8     public static void main(String[] args) {
9         // Create a fixed thread pool with 3 threads
10        ExecutorService executor = Executors.newFixedThreadPool(3);
11
12        // Submit 5 tasks to the pool
13        for (int i = 1; i <= 5; i++) {
14            Runnable task = new Task("Task " + i);
15            executor.execute(task);
16        }
17
18        // Shutdown the executor
19        executor.shutdown();
20    }
21 }
22
23 class Task implements Runnable {
24     private String name;
25
26     Task(String name) {
27         this.name = name;
28     }
29
30     @Override
31     public void run() {
32         System.out.println(name + " is running on " + Thread.currentThread().getName());
33         try {
34             // Simulate task execution time
35             Thread.sleep(2000);
36         } catch (InterruptedException e) {
37             Thread.currentThread().interrupt();
38         }
39         System.out.println(name + " completed.");
40     }
41 }

```

b) CachedThreadPool

A thread pool that creates new threads as needed but reuses previously constructed threads when available. Suitable for executing many short-lived tasks.

If all threads are busy, then create a new thread for the task and place it in the pool.

If thread is idle for 60 seconds (no task to execute) then kill the thread.

ExecutorService cachedThreadPool = Executors.newCachedThreadPool();

Example 2: Using CachedThreadPool

A cached thread pool is ideal for tasks that are short-lived and might vary in number.

Scenario: You have numerous short-lived tasks (e.g., handling web requests) that can be executed concurrently without worrying about the number of threads.

```

6 public class CachedThreadPoolExample {
7
8     public static void main(String[] args) {
9         // Create a cached thread pool
10        ExecutorService executor = Executors.newCachedThreadPool();
11
12        // Submit 10 short-lived tasks
13        for (int i = 1; i <= 10; i++) {
14            Runnable task = new Task("Task " + i);
15            executor.execute(task);
16        }
17
18        // Shutdown the executor
19        executor.shutdown();
20    }
21 }
22
23 class Task implements Runnable {
24     private String name;
25
26     Task(String name) {
27         this.name = name;
28     }
29
30     @Override
31     public void run() {
32         System.out.println(name + " is running on " + Thread.currentThread().getName());
33         // No sleep to simulate short-lived tasks
34         System.out.println(name + " completed.");
35     }
36 }

```

c) SingleThreadExecutor

A thread pool with only one thread. Tasks are executed sequentially, ensuring order and avoiding concurrency issues.

ExecutorService singleThreadExecutor = Executors.newSingleThreadExecutor();

Example 3: Using SingleThreadPool

A single-threaded executor executes tasks sequentially, ensuring that only one task is running at any given time.

```

1
2
3 *import java.util.concurrent.ExecutorService;
4
5
6 class Task4 implements Runnable
7 {
8     int number;
9     public Task4(int number) {
10        this.number = number;
11    }
12
13    @Override
14    public void run() {
15        System.out.println("Task" + number + " " + Thread.currentThread().getName() + ":" + number);
16        try{Thread.sleep(3000);}catch(Exception e) {}
17    }
18
19 }
20 public class SingleThreadPoolExample4 {
21
22     public static void main(String[] args) {
23         ExecutorService service = Executors.newSingleThreadExecutor();
24         for(int i=1; i<=10; i++) {
25             service.execute(new Task4(i));
26             try{Thread.sleep(3000);}catch(Exception e) {}
27         }
28         service.shutdown();
29     }
30 }

```

d) ScheduledThreadPool

A thread pool that can schedule commands to run after a given delay or to execute periodically.

ScheduledExecutorService scheduledThreadPool = Executors.newScheduledThreadPool(int corePoolSize);

Example 4: Using ScheduledThreadPool

This type of thread pool is used when you want to schedule tasks to execute after some delay or periodically.

Scenario: You need to schedule tasks to run after a delay or periodically (e.g., sending periodic reports).

```

3import java.util.concurrent.Executors;
4import java.util.concurrent.ScheduledExecutorService;
5import java.util.concurrent.TimeUnit;
6
7class Task5 implements Runnable
8{
9    int number;
10    public Task5(int number) {
11        this.number = number;
12    }
13
14    @Override
15    public void run() {
16        System.out.println("Task" + number + " " + Thread.currentThread().getName() + ":" + number);
17        try{Thread.sleep(3000);}catch(Exception e) {}
18    }
19
20}
21public class ScheduledThreadPoolExample5 {
22
23    public static void main(String[] args) {
24        ScheduledExecutorService service = Executors.newScheduledThreadPool(3);
25
26        for(int i=1; i<=10; i++) {
27            service.schedule(new Task5(i), 10, TimeUnit.SECONDS); // delay 10 seconds after previous task
28            service.scheduleAtFixedRate(new Task(i), 15, 10, TimeUnit.MILLISECONDS);
29            service.scheduleWithFixedDelay(new Task(i), 0, 15, TimeUnit.MILLISECONDS);
30            try{Thread.sleep(3000);}catch(Exception e) {}
31        }
32
33        service.shutdown();
34    }
35}

```

Advantages of Thread Pool and Executor Framework:

Better resource management: Avoids the overhead of creating and destroying threads repeatedly.

Improved performance: Tasks are executed efficiently using a fixed or dynamically growing number of threads.

Ease of use: With built-in classes and methods, Java's Executor framework simplifies the complexity of thread management.

Task Scheduling: The ScheduledExecutorService can easily schedule tasks for future execution.

Java Callable Interface:

Java provides two approaches for creating threads one by implementing the Runnable interface and the other by inheriting the Thread class. However, one important feature missing with the implementation of the Runnable interface is that it is not possible for a thread to return something when it completes its execution, i.e., when the run() method execution is over. In order to support this feature, the Java Callable interface is used.

Callable vs Runnable:

For implementing Runnable, the run() method needs to be implemented which does not return anything, while for a Callable, the call() method needs to be implemented which returns a result on completion. Note that a thread can't be created with a Callable, it can only be created with a Runnable.

Another difference is that the **call() method** can throw an exception whereas run() cannot

Syntax:

public Object call() throws Exception;

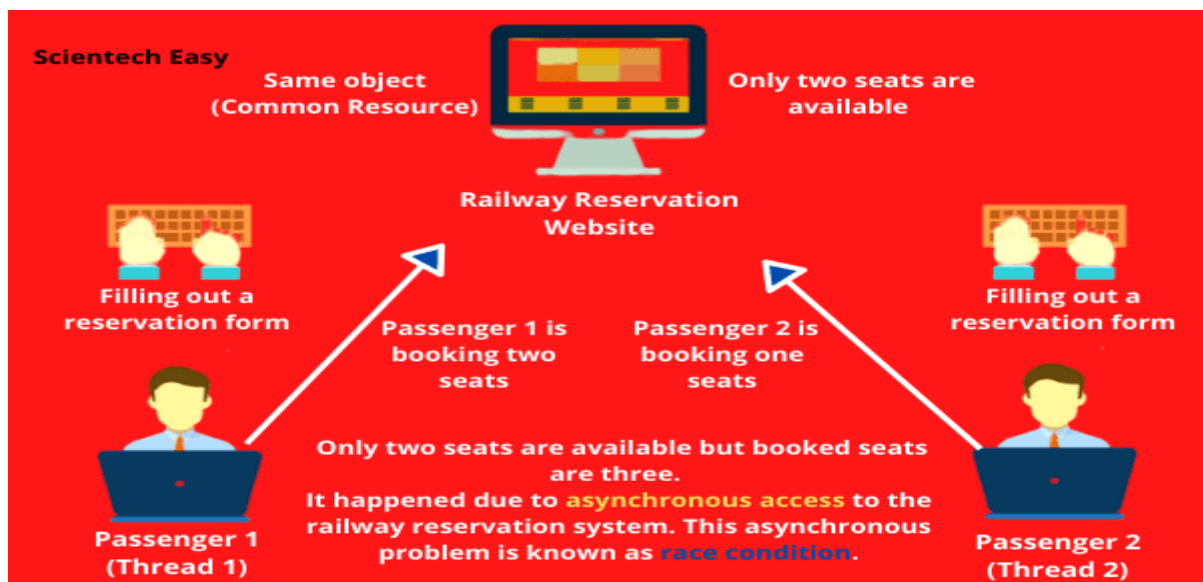
```

9 class CallableTask implements Callable<Integer>
10 {
11     int num, fact=1;
12     public CallableTask(int num) {
13         this.num = num;
14     }
15
16     @Override
17     public Integer call() throws Exception {
18         System.out.println("Name:" + Thread.currentThread().getName());
19         for(int i=1; i<=num; i++)
20         {
21             fact=fact*i;
22         }
23         return fact;
24     }
25 }
26
27 public class CallableExample6 {
28
29     public static void main(String[] args) throws InterruptedException, ExecutionException {
30         CallableTask c = new CallableTask(5);
31         ExecutorService service = Executors.newFixedThreadPool(2);
32         Future<Integer> res = service.submit(c);
33         System.out.println(res.get());
34     }
35 }

```

9.Thread Synchronization and Locks:

Problem with Multithreading in Java



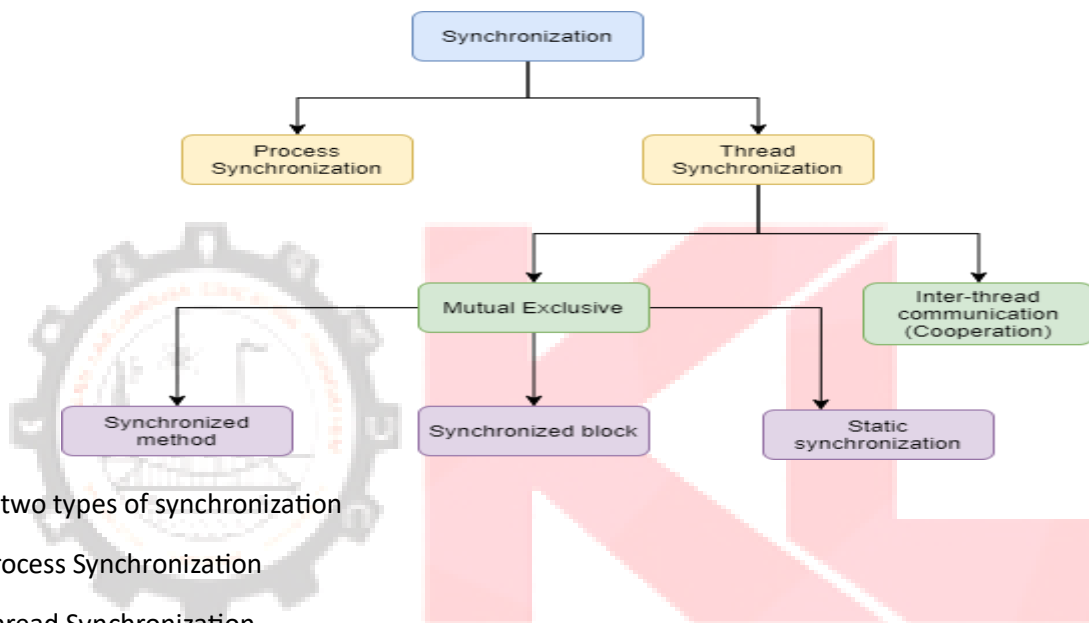
- Synchronization is essential when multiple threads access shared resources. Without synchronization, it may lead to data inconsistency.
- Synchronization in Java is the capability to control the access of multiple threads to any shared resource.
- Java Synchronization is a better option where we want to allow only one thread to access the shared resource.

Why use Synchronization?

The synchronization is mainly used to

1. To prevent thread interference.
2. To prevent consistency problem.

Types of Synchronization



There are two types of synchronization

1. Process Synchronization
2. Thread Synchronization

Thread Synchronization

There are two types of thread synchronization mutual exclusive and inter-thread communication.

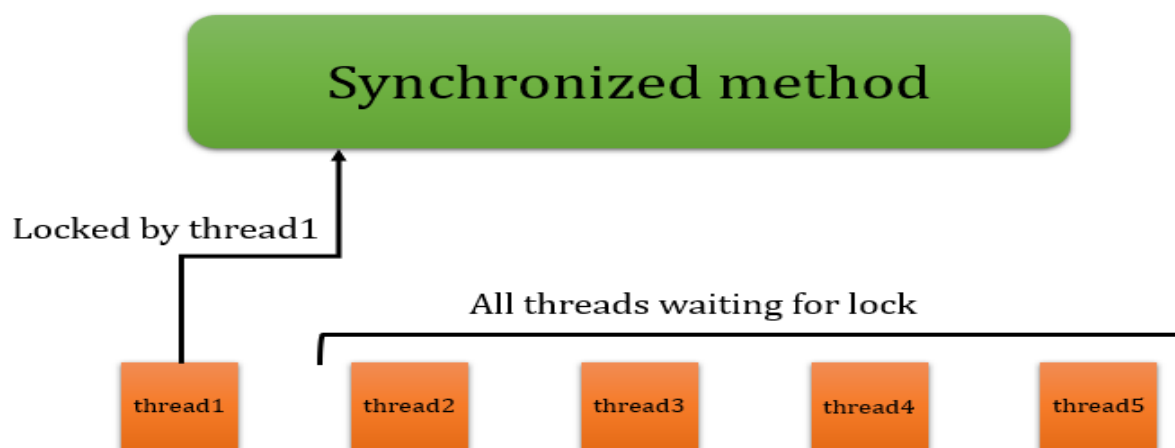
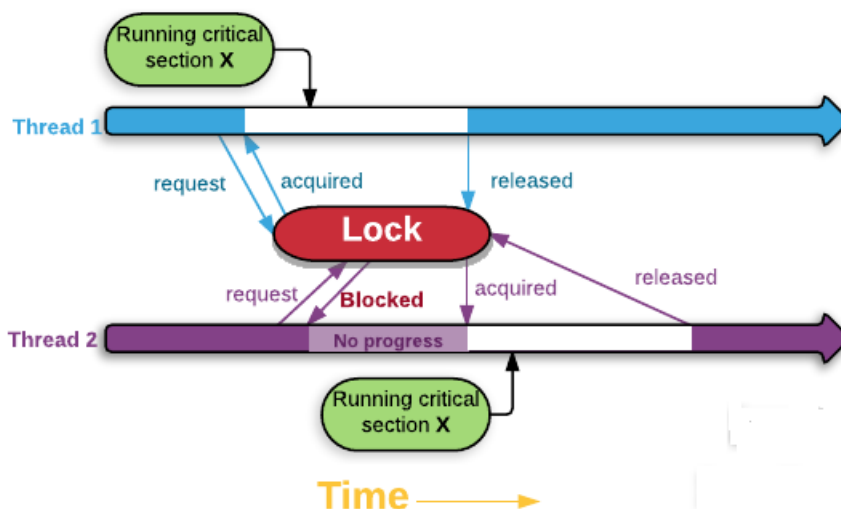
1. Mutual Exclusive
 1. Synchronized method.
 2. Synchronized block.
 3. Static synchronization.
2. Cooperation (Inter-thread communication in java)

Mutual Exclusive

Mutual Exclusive helps keep threads from interfering with one another while sharing data. It can be achieved by using the following three ways:

- i. By Using Synchronized Method
- ii. By Using Synchronized Block
- iii. By Using Static Synchronization

Mutual Exclusion of Critical Section



i). **Synchronized Method:** A method can be synchronized by using the synchronized keyword

```
public synchronized void methodName () {
    // synchronized code
}
```

Example of Synchronized Method:

```

3 class BookTicketSeat
4 {
5     int total_seats=10;
6     //synchronized method
7     synchronized void bookSeat(int seats)
8     {
9         if(total_seats >= seats)
10        {
11            System.out.println(seats + " Seats booked sucessfully.. " + "by " + Thread.currentThread().getName());
12            total_seats = total_seats-seats;
13            System.out.println(total_seats + " Seats available....");
14            System.out.println("Have a nice movie....");
15        }
16        else
17        {
18            System.out.println("Sorry, booking fail...");
19            System.out.println(total_seats + " Seats available....");
20            System.out.println("Try to next show....");
21        }
22    }
23 }
24
25 public class MovieTicketApp2 extends Thread {
26
27     protected static BookTicketSeat book;
28     int seats;
29     @Override
30     public void run() {
31         book.bookSeat(seats);
32     }
33
34     //main thread
35     public static void main(String[] args) {
36         book = new BookTicketSeat();
37
38         //create users (threads)
39         MovieTicketApp2 user1 = new MovieTicketApp2();
40         user1.seats=7;
41         user1.setName("Azmal");
42         user1.start();
43
44         MovieTicketApp2 user2 = new MovieTicketApp2();
45         user2.seats=6;
46         user2.setName("Mani");
47         user2.start();
48
49         MovieTicketApp2 user3 = new MovieTicketApp2();
50         user3.seats=2;
51         user3.setName("Bala");
52         user3.start();
53     }
54 }

```

ii). **Synchronized Block:** A block of code can be synchronized to allow only one thread to execute it at a time.

```

synchronized(object) {
    // synchronized code
}

```

Example of Synchronized block:

```

3 class BookTicketSeat
4 {
5     int total_seats=10;
6     void bookSeat(int seats)
7     {
8         //synchronized block
9         synchronized(this)
10        {
11            if(total_seats >= seats)
12            {
13                System.out.println(seats + " Seats booked sucessfully.. " + "by " + Thread.currentThread().getName());
14                total_seats = total_seats-seats;
15                System.out.println(total_seats + " Seats available....");
16                System.out.println("Have a nice movie....");
17            }
18            else
19            {
20                System.out.println("Sorry, seats are not available...");
21                System.out.println(total_seats + " Seats available....");
22                System.out.println("Try to next show....");
23            }
24        }
25    }
26 }
27
28 public class MovieTicketApp3 extends Thread {
29
30     protected static BookTicketSeat book;
31     int seats;
32     @Override
33     public void run() {
34         book.bookSeat(seats);
35     }
36 }
37
38 public static void main(String[] args) {
39     book = new BookTicketSeat();
40
41     //create users (threads)
42     MovieTicketApp3 user1 = new MovieTicketApp3();
43     user1.seats=7;
44     user1.setName("Azmal");
45     user1.start();
46
47     MovieTicketApp3 user2 = new MovieTicketApp3();
48     user2.seats=6;
49     user2.setName("Mani");
50     user2.start();
51
52     MovieTicketApp3 user3 = new MovieTicketApp3();
53     user3.seats=2;
54     user3.setName("Bala");
55     user3.start();
56 }
57 }

```

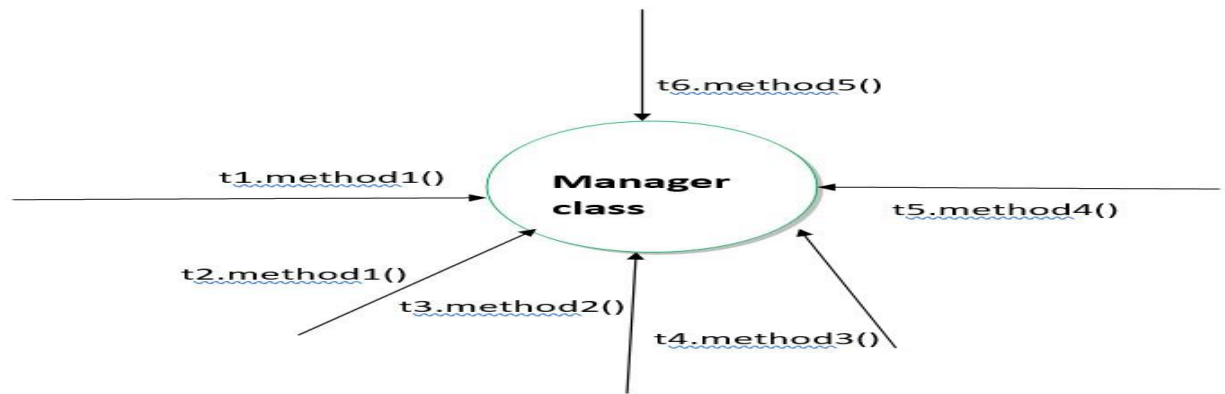
iii).Static synchronization: Static Synchronized method is also a method of synchronizing a method in Java such that no two threads can act simultaneously static upon the synchronized method. The only difference is by using Static Synchronized. We are attaining a class-level lock such that only one thread will operate on the method. The Thread will acquire a class-level lock of a Java class, such that only one thread can act on the static synchronized method.

Syntax:

```
synchronized static return type class name{}
```

Note: When a class has both synchronized and static synchronized methods they can run parallelly ,as those two methods require different locks.

Let us assume that there are 6 threads. The order of execution will be as follows:



Example of static synchronized:

```

3 class BookTicketSeat
4 {
5     static int total_seats=10;
6     //synchronized method
7     static synchronized void bookSeat(int seats)
8     {
9         if(total_seats >= seats)
10        {
11            System.out.println(seats + " Seats booked sucessfully.. " + "by " + Thread.currentThread().getName());
12            total_seats = total_seats-seats;
13            System.out.println(total_seats + " Seats available....");
14            System.out.println("Have a nice movie....");
15        }
16        else
17        {
18            System.out.println("Sorry, booking fail...");
19            System.out.println(total_seats + " Seats available....");
20            System.out.println("Try to next show....");
21        }
22    }
23 }
24
25 public class MovieTicketApp2 extends Thread {
26
27     protected static BookTicketSeat book;
28     int seats;
29     @Override
30     public void run() {
31         book.bookSeat(seats);
32     }
33
34     //main thread
35     public static void main(String[] args) {
36         book = new BookTicketSeat();
37
38         //create users (threads)
39         MovieTicketApp2 user1 = new MovieTicketApp2();
40         user1.seats=7;
41         user1.setName("Azmal");
42         user1.start();
43
44         MovieTicketApp2 user2 = new MovieTicketApp2();
45         user2.seats=6;
46         user2.setName("Mani");
47         user2.start();
48
49         MovieTicketApp2 user3 = new MovieTicketApp2();
50         user3.seats=2;
51         user3.setName("Bala");
52         user3.start();
53     }
54 }

```

Difference between Synchronization and static synchronization:

Synchronisation	Static Synchronisation
It needs the use of an object-level lock.	It needs the use of a class-level lock.
It is not necessary to declare its method as static.	Its method must be marked as static.
It is frequently used.	It isn't used on a regular basis.
For each object, a new instance is produced.	For the entire programme, there is only one instance.

LOCKS:

Synchronization is built around an internal entity known as the lock or monitor. Every object has a lock associated with it.

By convention, a thread that needs consistent access to an object's fields has to acquire the object's lock before accessing them, and then release the lock when it's done with them.

The lock() method

The lock() method is one of the most important methods of the Lock interface. It is used for acquiring the lock. For thread scheduling purposes, the current thread becomes disabled when the lock is not available. The lock() method is a public method that returns void.

Syntax:

```
public void lock()
```

The lockInterruptibly() method

The lockInterruptibly() method is another important method of Lock interface that is used for acquiring the lock unless the current thread is interrupted. It acquires and returns a lock immediately if it is available.

Syntax:

```
public void lockInterruptibly()
```

The tryLock() method

The tryLock() method is mainly used at the time of invocation for acquiring the lock. It returns the lock immediately with the Boolean value true when the lock is available. It returns the Boolean value false when the lock is not available.

The tryLock() method take no parameters and return the Boolean value.

Syntax:

```
public boolean tryLock()
```

10. Inter-thread Communication in Java:

- Inter-thread communication or Co-operation is all about allowing synchronized threads to communicate with each other.
- Inter-thread communication is important when you develop an application where two or more threads exchange some information.

Inter-thread Communication through wait() , notify() or notifyAll() methods:

Cooperation (Inter-thread communication) is a mechanism in which a thread is paused running in its critical section and another thread is allowed to enter (or lock) in the same critical section to be executed. It is implemented by following methods of Object class:

- wait()
- notify()
- notifyAll()

i) wait() method

The wait() method causes current thread to release the lock and wait until either another thread invokes the notify() method or the notifyAll() method for this object, or a specified amount of time has elapsed.

The current thread must own this object's monitor, so it must be called from the synchronized method only otherwise it will throw exception.

Method	Description
public final void wait()throws InterruptedException	It waits until object is notified.
public final void wait(long timeout)throws InterruptedException	It waits for the specified amount of time.

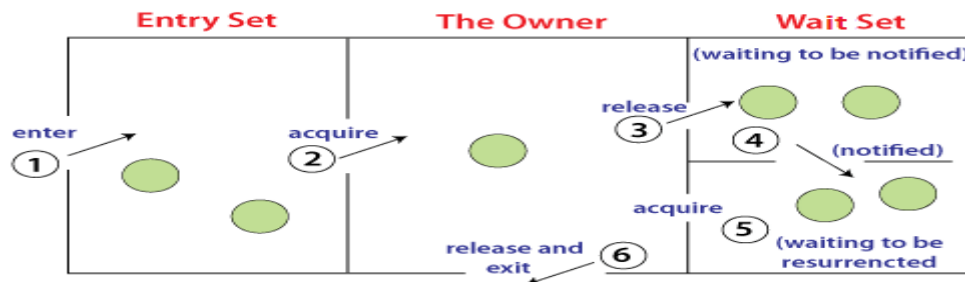
ii) notify() method

The notify() method wakes up a single thread that is waiting on this object's monitor. If any threads are waiting on this object, one of them is chosen to be awakened. The choice is arbitrary and occurs at the discretion of the implementation.

Syntax:

```
public final void notify()
```

Understanding the process of inter-thread communication:



- Threads enter to acquire lock.
- Lock is acquired by one thread.
- Now thread goes to waiting state if you call `wait()` method on the object. Otherwise it releases the lock and exits.
- If you call `notify()` or `notifyAll()` method, thread moves to the notified state (runnable state).
- Now thread is available to acquire lock.
- After completion of the task, thread releases the lock and exits the monitor state of the object.

Difference between wait and sleep:

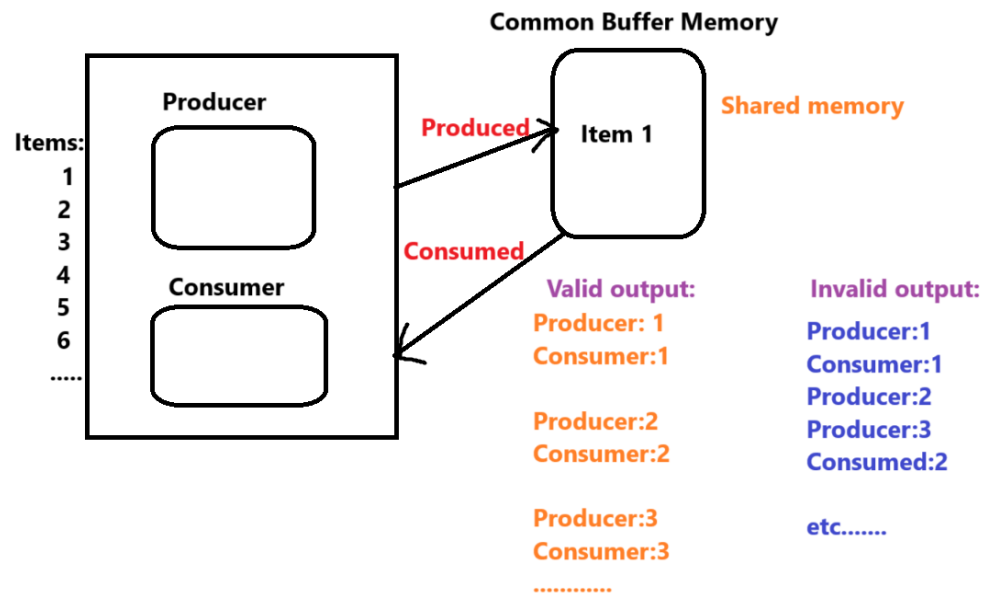
Wait() in Java	Sleep() in Java
It belongs to the object class.	It belongs to the thread class.
The lock on an object is released during synchronisation via the Wait() method.	The lock on an object isn't released when synchronising using the Sleep() method.
We can only call Wait() from the synchronised context.	We need not call Sleep() from the synchronised context.
The Wait() method is not static.	The Sleep() method is static.
The Wait() method consists of three overloaded methods, namely: • <code>wait (long timeout, int nanos)</code>. Here, nanos: Nanoseconds • <code>wait (long timeout)</code> • <code>wait()</code>	The Sleep() method consists of two overloaded methods, namely: • <code>sleep (long millis, int nanos)</code>. Here, nanos: Nanoseconds • <code>sleep (long millis)</code>. Here, millis: Milliseconds

Example of Interthread Communication:

Use Case:

Inter-thread communication:
Use Case1
Producer-Consumer Problem

Use case2:
Deposit and Withdraw



Note:

While Producer producing items then Consumer should not Consume the items and Vice versa.

At a time either Producer or Consumer have to be occupied shared memory.

USE CASE 1: Producer – Consumer problem using wait (), notify() and notifyAll() methods:

```

3 class Company
4 {
5     int num;
6     boolean valueSet=false; // if valueSet=false then chance of Producer otherwise Consumer chance
7     synchronized public void produce_item(int num)
8     {
9         if(valueSet)
10        {
11            try{wait();} catch(Exception e) {}
12        }
13        this.num=num;
14        System.out.println("Producer: " + this.num);
15        valueSet=true;
16        notify();
17    }
18 }
19 synchronized public int consume_item()
20 {
21     if(!valueSet)
22     {
23         try{wait();} catch(Exception e) {}
24     }
25     System.out.println("Consumed: " + this.num);
26     valueSet=false;
27     notify();
28     return this.num;
29 }
30 }
  
```

```

31 class Producer extends Thread
32 {
33     Company c;
34     public Producer(Company c)
35     {
36         this.c=c;
37     }
38     @Override
39     public void run() {
40         int i=1;
41         while(true) {
42             this.c.produce_item(i);
43             try {Thread.sleep(1000);} catch(Exception e) {}
44             i++;
45         }
46     }
47 }
48 class Consumer extends Thread
49 {
50     Company c;
51     public Consumer(Company c)
52     {
53         this.c=c;
54     }
55     @Override
56     public void run() {
57
58         while(true) {
59             this.c.consume_item();
60             try {Thread.sleep(1000);} catch(Exception e) {}
61         }
62     }}

```

```

63 public class InterThreadCommunicationDemo2 {
64
65     public static void main(String[] args) {
66         Company c = new Company();
67         Producer producer = new Producer(c);
68         producer.start();
69         Consumer consume = new Consumer(c);
70         consume.start();
71     }
72 }
73

```

Inter-Thread Communication through Blocking Queue Interface:

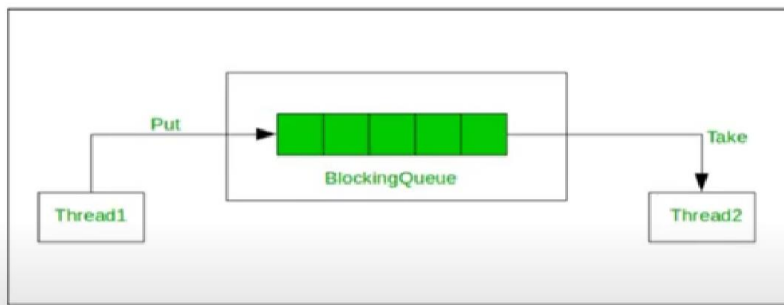
USE CASE 2: Producer-Consumer problem using BlockingQueue Interface:

Blocking Queue:

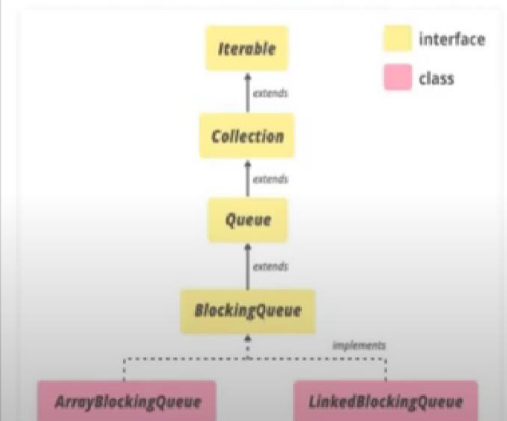
BlockingQueue is a queue that additionally supports operations that wait for the queue to become non-empty when we are trying to retrieve an element, and wait for the space to be empty when an element is to be inserted in the queue. Java 1.5 provides support for BlockingQueue interface along with other concurrent utility classes.

A BlockingQueue does not accept null elements. If we try to add a null value the implementation throws a `NullPointerException`.

Usage of BlockingQueue



The Hierarchy of BlockingQueue



BlockingQueue Types

1. Unbounded Queue:
2. Bounded Queue:

BlockingQueue Implementing Classes:

1. ArrayBlockingQueue
2. DelayQueue
3. LinkedBlockingDeque
4. LinkedBlockingQueue
5. LinkedTransferQueue
6. PriorityBlockingQueue
7. SynchronousQueue

Methods that block the operation

i. **put()**: The method inserts an element to the BlockingQueue. If in case the queue is full, the put() method waits until the queue has some vacant space to insert an element.

ii. **take()**: The method removes and returns an element from the BlockingQueue. If in case the queue is empty, the take() method waits until the queue has some elements to be deleted.

BlockingQueue Types

There are two types of BlockingQueue:

1. Unbounded Queue: Unbounded blocking queue is the queue that never blocks because its size could be grown to a very large size. The capacity of the BlockingQueue will be set to Integer.MAX_VALUE. When the elements are added, the size of the Unbounded queue grows.

Syntax :

```
BlockingQueue blockingQueue = new LinkedBlockingDeque( );
```

2. Bounded Queue: Another type of the blocking queue is the bounded queue. It can be created by passing the capacity of the queue to the constructor of the queue.

Syntax :

```
// Creating a Blocking Queue of the bounded nature with capacity 10
BlockingQueue blockingQueue = new LinkedBlockingDeque(10);
```

Example program:

```
3 import java.util.concurrent.ArrayBlockingQueue;
4 import java.util.concurrent.BlockingQueue;
5 import java.util.concurrent.LinkedBlockingQueue;
6
7 class Producer extends Thread
8 {
9     BlockingQueue<Integer> queue;
10    public Producer(BlockingQueue<Integer> queue)
11    {
12        this.queue=queue;
13    }
14    @Override
15    public void run() {
16        int i=1;
17        while(true)
18        {
19            try{this.queue.put(i);
20
21
22            System.out.println("Produced:" + i);
23            Thread.sleep(2000);}
24            catch(Exception e) {}
25            i++;
26        }
27    }
28 }
29
30 class Consumer extends Thread
31 {
32    BlockingQueue<Integer> queue;
33    public Consumer(BlockingQueue<Integer> queue)
34    {
35        this.queue=queue;
36    }
37    @Override
38    public void run() {
39        while(true)
40        {
41            try{int item = this.queue.take();
42
43            System.out.println("Consumed:" + item);
44            Thread.sleep(5000);}
45            catch(Exception e) {}
46        }
47    }
48 }
49 public class BlockingQueueExample {
50
51    public static void main(String[] args) {
52        BlockingQueue<Integer> queue = new ArrayBlockingQueue(5); //Bounded BlockingQueue
53        //BlockingQueue<Integer> queue = new LinkedBlockingQueue(); //UnBounded BlockingQueue
54        Producer produce = new Producer(queue);
55        produce.start();
56        Consumer consume = new Consumer(queue);
57        consume.start();
58    }
59 }
```

Semaphore in Java:

Thread Coordination through Semaphore in Java:

In multithreading, coordination between threads is crucial to avoid issues such as race conditions or deadlocks. A Semaphore in Java is used to control access to a resource by multiple threads. It allows a fixed number of threads to access a shared resource simultaneously. The Semaphore maintains a counter, which is decremented when a thread acquires the semaphore and incremented when a thread releases it.

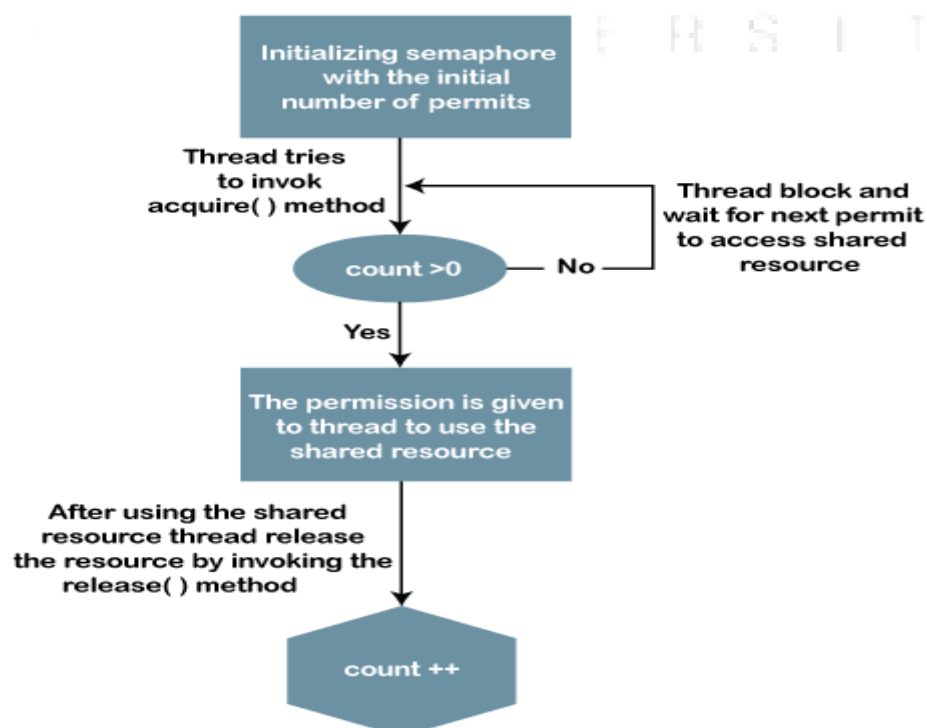
Working of Semaphore:

Acquire: When a thread wants to access a resource, it calls ***acquire()*** on the semaphore. If the semaphore's counter is greater than zero, it decrements the counter and allows the thread to proceed. If the counter is zero, the thread is blocked until another thread releases the semaphore.

Release: When a thread is done with the resource, it calls ***release()*** on the semaphore, which increments the counter and allows a blocked thread (if any) to acquire the semaphore.

Semaphore controls over the shared resource through a counter variable. The counter is a non-negative value. It contains a value either greater than 0 or equal to 0.

- If counter > 0, the thread gets permission to access the shared resource and the counter value is decremented by 1.
- Else, the thread will be blocked until a permit can be acquired.
- When the execution of the thread is completed then there is no need for the resource and the thread releases it. After releasing the resource, the counter value incremented by 1.
- If another thread is waiting for acquiring a resource, the thread will acquire a permit at that time.
- If counter = 0, the thread does not get permission to access the shared resource.



Working of Semaphore in Java

Types of Semaphores

In Java, there are two types of semaphores, based on their behavior and the way they manage permits:

1. Counting Semaphore (Unfair Semaphore):

- This is the default type of semaphore in Java, often referred to simply as Semaphore.
- It allows a set number of threads to access a shared resource concurrently.
- The order in which threads acquire permits is not guaranteed to be fair. A thread that requests a permit may obtain it before another thread that requested earlier, depending on the internal scheduling of the threads.

Key Characteristics:

- No guarantee of fairness.
- It is the most performant version, as it does not manage the order of waiting thread
- This is the default semaphore and is widely used for general-purpose scenarios.
- The order of acquiring permits is not guaranteed to be in any specific sequence, meaning that threads may get permits in a random or arbitrary order based on thread scheduling.
- It is ideal for scenarios where fairness is not a requirement, and the main goal is to optimize performance

Syntax:

```
Semaphore semaphore = new Semaphore(3); // Create an unfair semaphore with 3 permits
```

Example Program:

```
3 import java.util.concurrent.Semaphore;
4
5 public class CountingSemaphoreDemo {
6
7     public static void main(String[] args) {
8
9         Semaphore semaphore = new Semaphore(3); // Create an Unfair semaphore with 3 permits
10
11         //Anonymous Thread class
12         Runnable worker = new Runnable() {
13             @Override
14             public void run() {
15                 try {
16                     System.out.println(Thread.currentThread().getName() + " trying to acquire permit.");
17                     semaphore.acquire();
18                     System.out.println(Thread.currentThread().getName() + " acquired permit.");
19                     Thread.sleep(2000); // Simulate some work
20                 } catch (InterruptedException e) {
21                     e.printStackTrace();
22                 } finally {
23                     semaphore.release();
24                     System.out.println(Thread.currentThread().getName() + " released permit.");
25                 }
26             }
27         };
28
29         for (int i = 0; i < 5; i++) {
30             new Thread(worker, "Thread " + i).start();
31         }
32     }
33 }
```

2. Fair Semaphore:

- A fair semaphore ensures that threads acquire permits in the order they requested them, i.e., first-come, first-served (FIFO order).
- To create a fair semaphore, you pass true to the Semaphore constructor when creating it.

Syntax:

```
Semaphore semaphore = new Semaphore(3, true); // Create an fair semaphore with 3 permits
```

Key Characteristics:

- Ensures fairness: threads acquire permits in the order they requested them.

- Slightly slower compared to the default (unfair) semaphore due to the overhead of maintaining the fairness.

Example Program:

```

3 import java.util.concurrent.Semaphore;
4
5 public class CountingSemaphoreDemo {
6
7     public static void main(String[] args) {
8
9         Semaphore semaphore = new Semaphore(3, true); // Create an fair semaphore with 3 permits
10
11         //Anonymous Thread class
12         Runnable worker = new Runnable() {
13             @Override
14             public void run() {
15                 try {
16                     System.out.println(Thread.currentThread().getName() + " trying to acquire permit.");
17                     semaphore.acquire();
18                     System.out.println(Thread.currentThread().getName() + " acquired permit.");
19                     Thread.sleep(2000); // Simulate some work
20                 } catch (InterruptedException e) {
21                     e.printStackTrace();
22                 } finally {
23                     semaphore.release();
24                     System.out.println(Thread.currentThread().getName() + " released permit.");
25                 }
26             }
27         };
28
29         for (int i = 0; i < 5; i++) {
30             new Thread(worker, "Thread " + i).start();
31         }
32     }
33 }

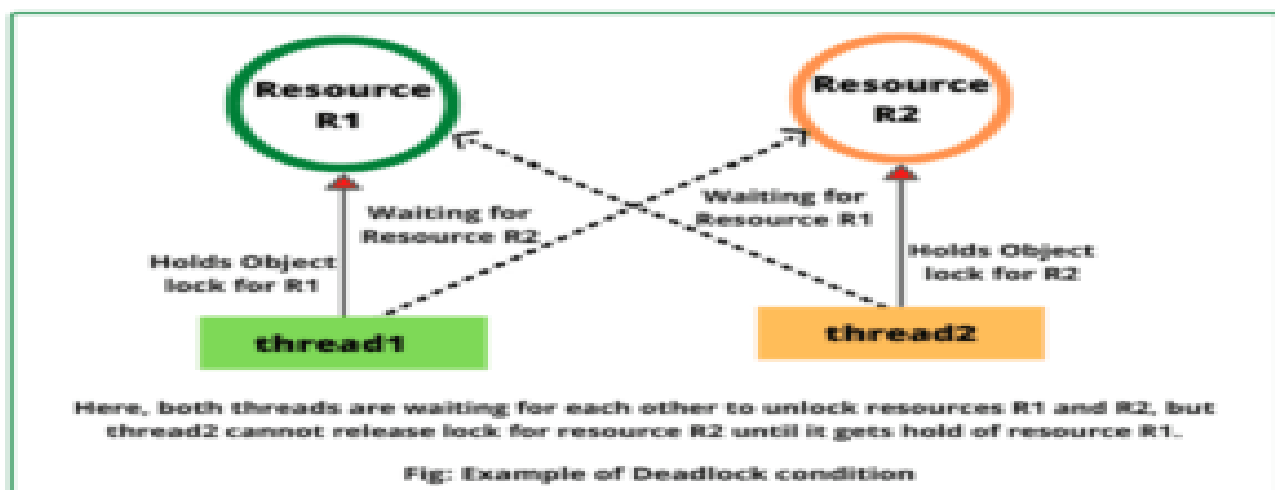
```

11. Deadlocks:

Deadlock in java is a programming situation where two or more threads are blocked forever. Java deadlock situation arises with at least two threads and two or more resources. Here I have written a simple program that will cause java deadlock scenario and then we will see how to analyse it.

synchronized keyword is used to make the class or method thread-safe which means only one thread can have the lock of the synchronized method and use it, other threads have to wait till the lock releases and anyone of them acquire that lock.

It is important to use if our program is running in a multi-threaded environment where two or more threads execute simultaneously. But sometimes it also causes a problem which is called Deadlock. Below is a simple example of the Deadlock condition.



DeadLock Example Program:

```

1
2
3 public class DeadLockDemo {
4
5     public static void main(String[] args) {
6         final String resource1 = "readingFile";
7         final String resource2 = "writingFile";
8         // t1 tries to lock resource1 then resource2
9         Thread t1 = new Thread() {
10            public void run() {
11                synchronized (resource1) {
12                    System.out.println("Thread 1: locked resource 1");
13
14                    try { Thread.sleep(100);} catch (Exception e) {}
15
16                    synchronized (resource2) {
17                        System.out.println("Thread 1: locked resource 2");
18                    }
19                }
20            }
21        };
22
23        // t2 tries to lock resource2 then resource1
24        Thread t2 = new Thread() {
25            public void run() {
26                synchronized (resource2) {
27                    System.out.println("Thread 2: locked resource 2");
28
29                    try { Thread.sleep(100);} catch (Exception e) {}
30
31                    synchronized (resource1) {
32                        System.out.println("Thread 2: locked resource 1");
33                    }
34                }
35            }
36        };
37
38        t1.start();
39        t2.start();
40
41    }
42
43 }

```

Detecting Deadlocks:

Deadlocks are hard to detect automatically, but Java provides some tools to help detect them:

1.jstack: This command-line tool shows the state of all threads in a JVM, including deadlocks. It is useful for analyzing thread dumps.

- To use jstack, find the process ID (PID) of your Java application and run
- The output will include information about any deadlocked threads.

```
jstack <PID>
```

2.VisualVM: This graphical tool monitors the JVM in real-time. It can analyze thread activity and detect deadlocks.

3.ThreadMXBean: You can use the ThreadMXBean class from the java.lang.management package to programmatically detect deadlocks.

Example Program:

```

6 public class DeadLockDemo {
7
8     public static void main(String[] args) {
9
10         ThreadMXBean bean = ManagementFactory.getThreadMXBean();
11         long[] deadlockedThreads = bean.findDeadlockedThreads();
12
13         if (deadlockedThreads != null) {
14             System.out.println("Deadlock detected!");
15         } else {
16             System.out.println("No deadlocks detected.");
17         }
18     }
19 }

```

Ways of avoid deadlock:

Although it is not possible to avoid deadlock condition but we can avoid it by using the following ways

- i). **Avoid Nested Locks** — You must avoid giving locks to multiple threads, this is the main reason for a deadlock condition. It normally happens when you give locks to multiple threads.
- ii). **Avoid Unnecessary Locks** — The locks should be given to the important threads. Giving locks to the unnecessary threads that cause the deadlock condition.
- iii). **Using Thread Join** — A deadlock usually happens when one thread is waiting for the other to finish. In this case, we can use Thread.join with a maximum time that a thread will take.

12. SYNCHRONIZED COLLECTIONS & PARALLEL PROGRAMMING:

Synchronized collections are designed to allow safe access and modification of data in a multi-threaded environment. By default, the collections in Java (like ArrayList, HashSet, HashMap, etc.) are not thread-safe. To avoid concurrency issues like data inconsistency and race conditions, the synchronized collections were introduced using methods from the **java.util.Collections** class.

The **Collections.synchronizedXXX()** methods provide a simple way to wrap non-synchronized collections like ArrayList, HashSet, and HashMap into synchronized versions.

Wrapper Methods:

Java provides several wrapper methods for synchronization:

1. Collections.synchronizedList()
2. Collections.synchronizedSet()
3. Collections.synchronizedMap()
4. Collections.synchronizedSortedMap()

5. Collections.synchronizedSortedSet()

These methods return a synchronized (thread-safe) version of the collection passed to them.

How It Works

When you use Collections.synchronizedXXX(), the resulting synchronized collection applies a lock on each method that modifies or accesses the collection. This ensures that only one thread can perform operations on the collection at any given time. However, it doesn't synchronize iterators, which requires additional steps to ensure safety during iteration.

Example Program: Collections.synchronizedList()

```

3 import java.util.Arrays;
4 import java.util.Collections;
5 import java.util.List;
6
7 public class SynchronizedCollectionDemo1 {
8
9     public static void main(String[] args) {
10         List<String> arrayList = Arrays.asList("Java", "C", "C++", "Oracle", "Python");
11         //Synchronized Collection
12         Collections.synchronizedList(arrayList);
13
14         //Create Thread1 - Anonymous class
15         Thread t1 = new Thread() {
16             @Override
17             public void run() {
18                 for(String list : arrayList) {
19                     System.out.println(Thread.currentThread().getName() + ":" + list);
20                 }
21             }
22         };
23         t1.start();
24
25         //Create Thread2 - Anonymous class
26         Thread t2 = new Thread() {
27             @Override
28             public void run() {
29                 for(String list : arrayList) {
30                     System.out.println(Thread.currentThread().getName() + ":" + list);
31                 }
32             }
33         };
34         t2.start();
35     }
36 }
37

```

Example Program : Collections.synchronizedSet()

```

9 public class SynchronizedCollectionDemo1 {
10
11     public static void main(String[] args) {
12         Set<String> set = new HashSet<String>();
13         set.add("Java");
14         set.add("C");
15         set.add("C++");
16         set.add("Python");
17
18         //Synchronized Collection
19         Collections.synchronizedSet(set);
20
21         //Create Thread1 - Anonymous class
22         Thread t1 = new Thread() {
23             @Override
24             public void run() {
25                 for(String list : set) {
26                     System.out.println(Thread.currentThread().getName() + ":" + list);
27                 }
28             }
29         };
30         t1.start();
31
32         //Create Thread2 - Anonymous class
33         Thread t2 = new Thread() {
34             @Override
35             public void run() {
36                 for(String list : set) {
37                     System.out.println(Thread.currentThread().getName() + ":" + list);
38                 }
39             }
40         };
41         t2.start();
42     }
43 }
44 }

```

Example Program : Collections.synchronizedMap()

```

11 public class SynchronizedCollectionDemo1 {
12
13     public static void main(String[] args) {
14         Map<Integer, String> map = new HashMap<Integer, String>();
15         map.put(1, "Java");
16         map.put(2, "C");
17         map.put(3, "C++");
18         map.put(4, "Python");
19
20         //Synchronized Collection
21         Collections.synchronizedMap(map);
22
23         //Create Thread1 - Anonymous class
24         Thread t1 = new Thread() {
25             @Override
26             public void run() {
27                 for (Map.Entry<Integer, String> entry : map.entrySet()) {
28                     System.out.println(entry.getKey() + ":" + entry.getValue());
29                 }
30             }
31         };
32         t1.start();
33
34         //Create Thread2 - Anonymous class
35         Thread t2 = new Thread() {
36             @Override
37             public void run() {
38                 for (Map.Entry<Integer, String> entry : map.entrySet()) {
39                     System.out.println(entry.getKey() + ":" + entry.getValue());
40                 }
41             }
42         };
43         t2.start();
44     }
45 }
46 }
47 }

```